Team Members: Afnan Ahmed, Abhinav Arabelly, Sanika Choudhary

The following work was done as part of my work at Flywl - cloud marketplace startup

FUID Pipeline

Date: JULY 2025 – OCT 2025

Problem Statement: To efficiently group product listings together under one unified identifier(FUID) across all marketplaces and to generate a workflow in cleaning bad marketplace data and improve matching.

What is FUID ? Why do we need it ?

FUID: is a unique identifier for a product across all marketplaces (GCP, AZURE, ISV) that may be present under different names across different paltforms so inconsistent naming causing delays in product discovery and matching workflows.

Dataset used : Product catalog data consisting of product information across 3 marketplaces

FUID Creation

Data Normalization - > FUID Creation (Structure) FUID – VENDOR

_

VERSION NUMBER

Vendor Name : Alphanumeric

NAME – PROUDCT ID –

FULL STACK WEB APPLICATION TO HOST THE PRODUCT CATALOG

FUID Pipeline: A full stack web-app made with React and Flask (Python) backend made for clients/customers to both register their products with FUIDs as well as search up relevant products with proper FUIDs

The Users for the Web application: Product team , Sales team

Features for the web application:1. FUID generation:

feature to register a product with a matching FUID

Input: Web app asks user for vendor, product, variation (optional), URL, long description, CSP(s)

Product Lookup: Based on user-input, tool checks whether the product already has a relevant ID and if so, returns the previously registered ID. If not, the user generates an FUID based on vendor, product name, and variation as well as logging the data into our database (JSON)

Finalized FUID: After the registration process, the user is able to see their corresponding FUID along with the extracted product info. This product info includes the inputted user data as well as any past data of various marketplace information and URLs

2. Product Catalog Search (FUID, product, vendor)

This is a search functionality where users can enter a product name, vendor name, or FUID, and the system retrieves and ranks relevant results based on match relevance and shows the following :

• Displays product name
• Displays vendor name
• Indicates platforms where the product is available (AWS, GCP, Azure)
• Provides links to the products
• Shows fuzzy match score
• FUID
• Lists category tags
• Developed Ask AI (RAG) capability on top of Fluids, enabling users to query product, vendor, and contract information using natural language

The search design considers fuzzy matching, embedding-based matching, and a weighted hybrid approach, with fuzzy matching currently used as it is more effective for shorter text inputs. We have also experimented with embedding-based and

hybrid approaches, but further evaluation is required to assess their effectiveness.
3. RAG (Intelligent chat-based responses)We integrated a RAG-based system into our platform, enabling intelligent, natural
language–driven interactions with product documents that contain long and detailed descriptions. It works by leveraging the existing long descriptions of products, and when multiple products have multiple descriptions, these are combined to generate comprehensive and context-aware responses, helping users gain insights into product offerings, features, and comparisons. Currently, each product has an individual Ask AI button that filters queries using product-specific metadata to answer only from that product's description, with future scope to extend this into a single unified AI interface that can answer queries across all products.

1. Chunking

We evaluated keyword chunking, recursive character splitting, and semantic chunking, where keyword chunking relies on predefined terms, semantic chunking adapts to topic shifts, and recursive character splitting preserves document structure by splitting hierarchically into paragraphs, sentences, words, and characters. Since our use case involved a dedicated Ask AI button per product (single-product context with no fixed keywords), recursive character splitting was the most suitable choice. A chunk size of 512 characters was selected because the documents were moderately sized, with an average length of approximately 6000 characters, ensuring balanced context retention and retrieval efficiency.

2. Vector Store

We used ChromaDB for local prototyping and initially experimented with Amazon OpenSearch Serverless, but due to high operational costs, we transitioned to an AWS S3–based vector store, achieving nearly 90% cost reduction. Cosine similarity was used for retrieval, while more advanced indexing techniques such as HNSW and IVF were identified as future optimization opportunities.

3. Retrieval

Keyword-based retrieval matches exact terms in queries, semantic retrieval captures contextual meaning using embeddings, and a hybrid approach combines both lexical and semantic strengths. All three approaches were evaluated on a synthetically generated validation dataset, and the hybrid retrieval method performed best based on evaluation metrics such as precision and recall.

4. Re-ranking

Re-ranking is essential in RAG pipelines to refine retrieved results and surface the most contextually relevant documents before generation, thereby improving overall answer quality. For our use case, we applied a contextual compression re-ranker, which enhanced precision by prioritizing the most relevant chunks and improving retriever effectiveness.

5. Generation

We experimented with multiple OpenAI generation models, including GPT-4, GPT-3.5, and GPT-3.5 Mini, and evaluated them against benchmark criteria for instruction-following and response quality. The focus was on minimizing hallucinations while ensuring accurate, grounded, and instruction-compliant outputs.

6. Evaluation and Observability

We used the RAGAS framework for evaluation and generated a synthetic test dataset using LLMs, with plans to replace it with real user queries and ground-truth answers sourced from product and sales teams. Retrieval was evaluated using precision, recall, and Mean Reciprocal Rank (MRR), while generation quality was measured using faithfulness, accuracy, and relevance. Additionally, non-functional metrics such as latency and cost per query were tracked to assess system efficiency and trade-offs. We used Lang Chain framework to implement and Lang smith for observability.

AWS Deployment

The full-stack web application was deployed using AWS services including EC2 for compute, S3 for storage and vector data, IAM for access management, Application Load Balancer (ALB) for traffic handling

As part of Flywl training program

GCP CertificationsDate: NOV 2025

1. Sanika: I completed the Google Cloud Data Practitioner certification, building a strong foundation in data services, analytics workflows, and cloud data fundamentals on GCP. I also gained practical understanding of data ingestion, storage, analysis, and concepts relevant to real-world data platforms.
2. Afnan: Completion of the GCP ACE Certificate
a. Overall Experience: I really enjoyed learning how to use the GCP environment. As we previously used AWS, it was interesting to see how another platform is used to deploy applications and store data on a higher level for thousands of users. Talking with others and going through various practice exams, I gained practical experience and can, in a sense, work as a cloud engineer as well.
3. Abhinav: I completed the Google Cloud Machine Learning certification, gaining a strong foundation in designing, building, and deploying machine learning models on Google Cloud Platform. The certification covered core ML concepts, data preparation, model training and evaluation, as well as the use of GCP services such as Vertex AI, AutoML, and BigQuery ML. I also developed a practical understanding of implementing end-to-end ML workflows, from feature engineering to model deployment and monitoring in real-world production environments.

LLM based product matching at Flywl

Product Matching Workflow (csv) - Jacobo

Date: DEC 2025 – JAN 2025
Objective: To make an efficient matching pipeline when a csv file is uploaded containing multiple products and vendors to allow the Sales Team to make faster transactions and matches for clients/customers. In practice, teams often have large spreadsheets of customer subscriptions, but no clear way to tell:
• Which vendors are actually available on a marketplace
• Which specific product listings match their internal software names
• Whether an AWS product qualifies for enterprise commit usage
Our objective was to turn that messy input into a clear, actionable view that shows marketplace eligibility and matching confidence.
Internal Problem Statement: The Sales Team spends excess time manually matching client data to our product catalog in determining renewals or relevant matches. Can an automated pipeline fasten this process?
How the solution works (at a high level)
We built an interactive dashboard that allows a user to upload a customer subscription inventory and automatically get enriched marketplace matching results.
The flow is intentionally simple from a user's point of view:
1. 2. 3. 4. Upload a CSV of vendor + software names
Adjust matching sensitivity if needed
Review marketplace availability and product matches
Export the results for downstream analysis or customer conversations
Behind the scenes, the system handles normalization, matching, and enrichment in multiple stages to maximize accuracy.

Structure:Cleaning
Before any matching happens, we normalize both vendor and product names so that comparisons are meaningful. This includes:
• Cleaning whitespace and punctuation
• Lowercasing
• Removing noise characters and formatting artifacts
• Applying the same normalization rules consistently across customer data, marketplace listings, and AI-generated outputs
This step dramatically improves matching quality and reduces false negatives.
Trial: We did attempt to remove extra (common) company additions such as inc., corp., corporations, etc. using the cleanco library which led to inaccuracy in matching our data to the catalog. Therefore, we removed this feature.
Vendor matching approach
Vendor matching happens in stages, progressing from fast and deterministic to more flexible and intelligent only when needed.
Stage 1: Direct vendor matching
We first try to match the customer's vendor name directly against known marketplace vendors using fuzzy matching. If the similarity score is high enough, we accept the match immediately.
Stage 2: Variation-based matching
If a direct match fails, we look up the vendor name in our variations knowledge base. Many vendors appear under different names internally (for example abbreviations, product names, or regional labels).
When a variation maps to a known canonical vendor, we retry the marketplace match using that canonical name.
Stage 3: AI-assisted matching (optional)
When both earlier stages fail, we optionally use an LLM to:
• Clean and normalize the vendor name into a likely canonical form• Generate realistic name variations a human might use
Each AI-generated variation is then matched against marketplace vendors.
This stage is intentionally optional due to cost and latency, and is used only when needed.
If none of the stages produce a confident match, the vendor is marked as unavailable on that marketplace.
Product matching approach
Once a vendor is matched for a given marketplace, we then attempt to match the software/product name.
For the matched vendor:
• We compare the customer's software name against that vendor's marketplace product listings
• We select the best-scoring product if it exceeds a configurable confidence threshold
For AWS specifically, we also check whether the matched product is eligible for enterprise commits and surface that as a separate signal.
If no product clears the threshold, we still retain the vendor match but mark the product as unmatched.
What the dashboard shows
For each customer subscription, the dashboard produces:
• Whether the vendor is available on AWS, GCP, and/or Azure
• Which vendor and product were matched on each marketplace
• Confidence scores for both vendor and product matches
• Whether an AWS product is enterprise-commit eligible
• Whether any marketplace match exists at all
This gives teams both a yes/no answer and the confidence context needed to trust the

result.Workflow:
THANKYOU NOTE
Linked Docs:
Large Documentation.docx
Workflow
AWS Marketplace Max - AgentGCP - ACE: Certificate Notes
Product Integration Notes.docx
AWS Marketplace Max - Test Prompt Cases.docx
Opensearch + Bedrock Notes Meeting.docx